DRDD API Documentation

Versão: 1.0 **Framework:** FastAPI (Python) **Autenticação:** OAuth2 com JWT **Base URL:** `http://localhost:8000`

Índice

1. Autenticação
2. Rotas Gerais
3. Rotas de Usuário (Common)
4. Rotas de Staff
5. Rotas de Estabelecimento
6. Códigos de Erro
7. Modelos de Dados

Autenticação

1. Login

Endpoint: `POST /login`

Descrição: Autentica o usuário e retorna um token JWT

Request:

```
{
  "username": "string",
  "password": "string"
}
```

Response (200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

Erros:

- `401 Unauthorized` - Usuário ou senha inválidos

Notas:

- Token válido por 30 minutos

- Usar token no header: `Authorization: Bearer <token>`

Rotas Gerais

1. Health Check

Endpoint: `GET /`

Descrição: Verifica se a API está funcionando

Response (200 OK):

```
{
  "message": "API funcionando □"
}
```

Rotas de Usuário (Common)

1. Obter Dados do Usuário Atual

Endpoint: `GET /users/common`

Descrição: Retorna os dados completos do usuário autenticado

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "user_type": "COMMON",
  "address": null,
  "role": null
}
```

Erros:

- `400 Bad Request` - Erro ao buscar dados
- `401 Unauthorized` - Token inválido ou expirado
- `500 Internal Server Error` - Erro interno

---

2. Criar Novo Usuário

Endpoint: `POST /users/common`

Descrição: Cria um novo usuário comum

Request:

```
{
  "username": "string",
  "email": "email@example.com",
  "password": "string"
}
```

Validações de Senha:

- Mínimo 8 caracteres
- Pelo menos 1 letra maiúscula (A-Z)
- Pelo menos 1 letra minúscula (a-z)
- Pelo menos 1 número (0-9)
- Pelo menos 1 caractere especial (!@#$%^&*(),.?":{}|<>)

Response (201 Created):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "user_type": "COMMON"
}
```

Erros:

- `400 Bad Request` - Dados inválidos ou senha não atende validações
- `500 Internal Server Error` - Erro interno

---

3. Atualizar Dados do Usuário

Endpoint: `PUT /users/common`

Descrição: Atualiza os dados do usuário autenticado

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Request:

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "password_hash": "string",
  "user_type": "COMMON"
}
```

Response (200 OK):

```
{
  "message": "Usuário atualizado com sucesso"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

---

4. Marcar Prato como Favorito

Endpoint: `POST /dishes/like/{dish_id}`

Descrição: Adiciona um prato à lista de favoritos do usuário

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Path Parameters:

- `dish_id` (string, obrigatório): UUID do prato

Response (201 Created):

```
{
  "message": "Prato adicionado aos favoritos"
}
```

Erros:

- `400 Bad Request` - Prato inválido
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

---

5. Remover Prato dos Favoritos

Endpoint: `DELETE /dishes/dislike/{dish_id}`

Descrição: Remove um prato da lista de favoritos

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Path Parameters:

- `dish_id` (string, obrigatório): UUID do prato

Response (204 No Content):

```

```

Erros:

- `400 Bad Request` - Prato inválido
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

---

6. Obter Pratos Favoritos

Endpoint: `GET /dishes/liked`

Descrição: Retorna todos os pratos marcados como favoritos pelo usuário

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Response (200 OK):

```
{
  "dishes": [
    {
      "id": "uuid-string",
```

```
      "stablishment_id": "uuid-string",
      "name": "string",
      "dish_image_path": "string",
      "associated_entry": "uuid-string"
    }
  ]
}
```

Erros:

- `400 Bad Request` - Erro ao buscar favoritos
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

---

7. Criar Reclamação

Endpoint: `POST /complaints`

Descrição: Submete uma reclamação sobre um estabelecimento

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Request:

```
{
  "establishment_id": "uuid-string",
  "title": "string",
  "complaint_text": "string"
}
```

Response (201 Created):

```
{
  "id": "uuid-string",
  "establishment_id": "uuid-string",
  "title": "string",
  "complaint_text": "string",
  "user_id": "uuid-string",
  "created_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

---

8. Obter Minhas Reclamações

Endpoint: `GET /complaints`

Descrição: Retorna todas as reclamações criadas pelo usuário autenticado

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Response (200 OK):

```
{
  "complaints": [
    {
      "id": "uuid-string",
      "establishment_id": "uuid-string",
      "title": "string",
      "complaint_text": "string",
      "created_at": "2024-01-01T10:00:00Z",
      "status": "pending"
    }
  ]
}
```

Erros:

- `400 Bad Request` - Erro ao buscar reclamações
- `401 Unauthorized` - Não autenticado
- `500 Internal Server Error` - Erro interno

Rotas de Staff

⚠ **Nota de Permissão:** Todas as rotas de staff requerem `user_type = "STAFF"`

1. Obter Dados de Outro Usuário

Endpoint: `GET /staff/user/{user_to_get}`

Descrição: Staff pode visualizar dados de outro usuário

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `user_to_get` (string, obrigatório): UUID do usuário a consultar

Response (200 OK):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "user_type": "COMMON"
}
```

Erros:

- `400 Bad Request` - Usuário inválido
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

2. Criar Novo Usuário (Staff)

Endpoint: `POST /staff/user`

Descrição: Staff cria novo usuário

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "username": "string",
  "email": "email@example.com",
  "password": "string"
}
```

Response (201 Created):

```
{
  "id": "uuid-string",
```

```
  "username": "string",
  "email": "email@example.com",
  "user_type": "COMMON"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

3. Deletar Usuário (Staff)

Endpoint: `DELETE /staff/user/{user_to_delete}`

Descrição: Staff deleta um usuário

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `user_to_delete` (string, obrigatório): UUID do usuário a deletar

Response (204 No Content):

```

```

Erros:

- `400 Bad Request` - Usuário inválido
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

4. Obter Reclamações de um Usuário

Endpoint: `GET /staff/complaints/{user_id}`

Descrição: Staff visualiza reclamações feitas por um usuário específico

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `user_id` (string, obrigatório): UUID do usuário

Response (200 OK):

```
{
  "complaints": [
    {
      "id": "uuid-string",
      "establishment_id": "uuid-string",
      "title": "string",
      "complaint_text": "string",
      "user_id": "uuid-string",
      "created_at": "2024-01-01T10:00:00Z",
      "status": "pending"
    }
  ]
}
```

Erros:

- `400 Bad Request` - Usuário inválido
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

5. Validar/Responder Reclamação

Endpoint: `POST /staff/complaints`

Descrição: Staff valida e responde uma reclamação

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "complaint_id": "uuid-string",
  "validation": true,
  "establishment_id": "uuid-string",
  "complaint_response": "string"
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "status": "validated",
  "response": "string",
  "responded_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

6. Deletar Reclamação

Endpoint: `DELETE /staff/complaints/{complaint_id}`

Descrição: Staff deleta uma reclamação

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `complaint_id` (string, obrigatório): UUID da reclamação

Response (200 OK):

```
{
  "message": "Reclamação deletada com sucesso"
}
```

Erros:

- `400 Bad Request` - Reclamação inválida
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

7. Obter Entrada Específica

Endpoint: `GET /staff/entry/{entry}`

Descrição: Staff obtém informações de uma entrada específica

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `entry` (EntryCreator): Objeto com dados da entrada

Response (200 OK):

```
{
  "id": "uuid-string",
  "name": "string",
  "picture": "string",
  "audio": "string",
  "entry_text": "string",
  "category": "simples",
  "estimated_time": "string",
  "difficulty_level": "facil"
}
```

Erros:

- `400 Bad Request` - Entrada inválida
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

8. Obter Todas as Entradas

Endpoint: `GET /staff/entry/all`

Descrição: Staff obtém lista de todas as entradas

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Response (200 OK):

```
{
  "entries": [
    {
      "id": "uuid-string",
      "name": "string",
      "picture": "string",
```

```
      "audio": "string",
      "entry_text": "string",
      "category": "simples",
      "estimated_time": "string",
      "difficulty_level": "facil"
    }
  ]
}
```

Erros:

- `400 Bad Request` - Erro ao buscar entradas
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

9. Criar Entrada

Endpoint: `POST /staff/entry`

Descrição: Staff cria nova entrada

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "id": "uuid-string",
  "name": "string",
  "picture": "string",
  "audio": "string",
  "entry_text": "string",
  "category": "simples",
  "estimated_time": "string",
  "difficulty_level": "facil"
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "name": "string",
  "created_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

10. Deletar Entrada

Endpoint: `DELETE /staff/entry/{entry_id}`

Descrição: Staff deleta uma entrada

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `entry_id` (string, obrigatório): UUID da entrada

Response (204 No Content):

```

```

Erros:

- `400 Bad Request` - Entrada inválida
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

11. Atualizar Entrada

Endpoint: `PUT /staff/entry`

Descrição: Staff atualiza uma entrada

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "id": "uuid-string",
```

```
  "name": "string",
  "picture": "string",
  "audio": "string",
  "entry_text": "string",
  "category": "simples",
  "estimated_time": "string",
  "difficulty_level": "facil"
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "name": "string",
  "updated_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

12. Obter Todos os Eventos

Endpoint: `GET /staff/event/all`

Descrição: Staff obtém lista de todos os eventos

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Response (200 OK):

```
{
  "events": [
    {
      "id": "uuid-string",
      "name": "string",
      "event_date": "2024-01-01",
      "description": "string",
      "address": {
        "city": "string",
        "street": "string",
        "neighborhood": "string",
```

```
          "zip_code": "string"
        }
      }
    ]
  }
```

Erros:

- `400 Bad Request` - Erro ao buscar eventos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

13. Obter Evento Específico

Endpoint: `GET /staff/event/{event_id}`

Descrição: Staff obtém informações de um evento específico

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `event_id` (string, obrigatório): UUID do evento

Response (200 OK):

```
{
  "id": "uuid-string",
  "name": "string",
  "event_date": "2024-01-01",
  "description": "string",
  "address": {
    "city": "string",
    "street": "string",
    "neighborhood": "string",
    "zip_code": "string"
  }
}
```

Erros:

- `400 Bad Request` - Evento inválido
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

14. Criar Evento

Endpoint: `POST /staff/event`

Descrição: Staff cria novo evento

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "id": "uuid-string",
  "name": "string",
  "event_date": "2024-01-01",
  "description": "string",
  "address": {
    "city": "string",
    "street": "string",
    "neighborhood": "string",
    "zip_code": "string"
  }
}
```

Response (201 Created):

```
{
  "id": "uuid-string",
  "name": "string",
  "created_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

15. Atualizar Evento

Endpoint: `PUT /staff/event`

Descrição: Staff atualiza um evento

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Request:

```
{
  "id": "uuid-string",
  "name": "string",
  "event_date": "2024-01-01",
  "description": "string",
  "address": {
    "city": "string",
    "street": "string",
    "neighborhood": "string",
    "zip_code": "string"
  }
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "name": "string",
  "updated_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

---

16. Deletar Evento

Endpoint: `DELETE /staff/event/{event_id}`

Descrição: Staff deleta um evento

Headers Obrigatórios:

```
Authorization: Bearer <token> (Staff)
```

Path Parameters:

- `event_id` (string, obrigatório): UUID do evento

Response (204 No Content):

```
```

Erros:

- `400 Bad Request` - Evento inválido
- `401 Unauthorized` - Não autenticado ou não é Staff
- `500 Internal Server Error` - Erro interno

Rotas de Estabelecimento

⚠ **Nota de Permissão:** Requer `user_type = "ESTABLISHMENT"` (quando especificado)

1. Criar Estabelecimento

Endpoint: `POST /establishments`

Descrição: Cria novo estabelecimento

Request:

```
{
  "username": "string",
  "email": "email@example.com",
  "password": "string",
  "address": "string"
}
```

Response (201 Created):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "address": "string",
  "user_type": "ESTABLISHMENT"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `500 Internal Server Error` - Erro interno

---

2. Atualizar Estabelecimento

Endpoint: `PUT /establishments`

Descrição: Atualiza dados do estabelecimento

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Request:

```
{
  "username": "string",
  "email": "email@example.com",
  "address": "string"
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "address": "string",
  "user_type": "ESTABLISHMENT"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou não é estabelecimento
- `500 Internal Server Error` - Erro interno

---

3. Obter Dados do Estabelecimento

Endpoint: `GET /establishments`

Descrição: Retorna dados do estabelecimento autenticado

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "username": "string",
  "email": "email@example.com",
  "address": "string",
  "user_type": "ESTABLISHMENT"
}
```

Erros:

- `400 Bad Request` - Erro ao buscar estabelecimento
- `401 Unauthorized` - Não autenticado ou usuário é comum
- `500 Internal Server Error` - Erro interno

---

4. Obter Pratos do Estabelecimento

Endpoint: `GET /establishments/dish`

Descrição: Retorna todos os pratos postados pelo estabelecimento autenticado

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Response (200 OK):

```
{
  "dishes": [
    {
      "id": "uuid-string",
      "stablishment_id": "uuid-string",
      "name": "string",
      "dish_image_path": "string",
      "associated_entry": "uuid-string"
    }
  ]
}
```

Erros:

- `400 Bad Request` - Erro ao buscar pratos
- `401 Unauthorized` - Não autenticado ou usuário é comum
- `500 Internal Server Error` - Erro interno

---

5. Criar Prato

Endpoint: `POST /establishments/dish`

Descrição: Cria novo prato para o estabelecimento

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Request:

```
{
  "stablishment_id": "uuid-string",
  "name": "string",
  "dish_image_path": "string",
  "associated_entry": "uuid-string"
}
```

Response (201 Created):

```
{
  "id": "uuid-string",
  "stablishment_id": "uuid-string",
  "name": "string",
  "dish_image_path": "string",
  "associated_entry": "uuid-string"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou usuário é comum
- `500 Internal Server Error` - Erro interno

---

6. Atualizar Prato

Endpoint: `PUT /establishments/dish`

Descrição: Atualiza um prato existente

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Request:

```
{
  "id": "uuid-string",
  "stablishment_id": "uuid-string",
  "name": "string",
  "dish_image_path": "string",
  "associated_entry": "uuid-string"
}
```

Response (200 OK):

```
{
  "id": "uuid-string",
  "stablishment_id": "uuid-string",
  "name": "string",
  "dish_image_path": "string",
  "associated_entry": "uuid-string",
  "updated_at": "2024-01-01T10:00:00Z"
}
```

Erros:

- `400 Bad Request` - Dados inválidos
- `401 Unauthorized` - Não autenticado ou usuário é comum
- `500 Internal Server Error` - Erro interno

---

7. Deletar Prato

Endpoint: `DELETE /establishments/dish/{dish_id}`

Descrição: Deleta um prato do estabelecimento

Headers Obrigatórios:

```
Authorization: Bearer <token>
```

Path Parameters:

- `dish_id` (string, obrigatório): UUID do prato

Response (204 No Content):

```

```

Erros:

- `400 Bad Request` - Prato inválido
- `401 Unauthorized` - Não autenticado ou usuário é comum
- `500 Internal Server Error` - Erro interno

Códigos de Erro

Código	Descrição	Causa Comum
`200 OK`	Sucesso	Operação concluída
`201 Created`	Recurso criado	Novo registro adicionado
`204 No Content`	Sucesso (sem resposta)	Deleção bem-sucedida
`400 Bad Request`	Dados inválidos	Validação falhou, campos obrigatórios ausentes
`401 Unauthorized`	Não autenticado	Token inválido, expirado ou ausente
`500 Internal Server Error`	Erro interno	Erro no servidor

Modelos de Dados

UserFullResponse

```
{
  "id": "uuid-string ou int",
  "username": "string",
  "email": "email@example.com",
  "user_type": "COMMON | STAFF | ESTABLISHMENT",
  "address": "string | null",
  "role": "string | null"
}
```

DishResponse

```
{
  "id": "uuid-string",
  "stablishment_id": "uuid-string",
  "name": "string",
  "dish_image_path": "string",
  "associated_entry": "uuid-string"
}
```

EntryCreator

```
{
  "id": "uuid-string",
  "name": "string",
  "picture": "string",
  "audio": "string",
  "entry_text": "string",
  "category": "simples",
  "estimated_time": "string",
  "difficulty_level": "facil | moderado | dificil"
}
```

EventCreator

```
{
  "id": "uuid-string",
  "name": "string",
  "event_date": "YYYY-MM-DD",
  "description": "string",
  "address": {
    "city": "string",
    "street": "string",
    "neighborhood": "string",
    "zip_code": "string"
  }
}
```

ComplaintValidator

```
{
  "complaint_id": "uuid-string",
  "validation": "boolean",
  "establishment_id": "uuid-string",
  "complaint_response": "string"
}
```

Address

```
{
  "city": "string",
  "street": "string",
  "neighborhood": "string",
  "zip_code": "string"
}
```

Configurações Importantes

- **Porta padrão:** 8000
- **Secret Key JWT:** `sua_chave_super_secreta_aqui` (⚠ Alterar em produção!)
- **Algoritmo JWT:** HS256
- **Expiração de Token:** 30 minutos
- **Database:** PostgreSQL (SQLModel/SQLAlchemy)

Notas para Integração Frontend

1. **Autenticação:**
    - Fazer login primeiro via `/login`
    - Guardar `access_token` no localStorage ou sessionStorage
    - Enviar token em todo request autenticado no header: `Authorization: Bearer <token>`
2. **Tipos de Usuário:**
    - `COMMON` - Usuário comum (não pode acessar rotas de staff/establishment)
    - `STAFF` - Gerenciador (acesso a rotas de staff)
    - `ESTABLISHMENT` - Estabelecimento (pode gerenciar pratos e dados)
3. **Validação de Senha:**
    - Implementar validação no frontend antes de enviar
    - Exibir mensagens claras de erro para cada critério
4. **CORS:**
    - Verificar se CORS está habilitado na API para o domínio do frontend
5. **Endpoints Sem Autenticação:**
    - `GET /` - Health check
    - `POST /login` - Login
    - `POST /establishments` - Criar estabelecimento
    - Todos os outros requerem token válido
6. **Issues Encontradas no Código:**
    - `UserCreator` é importado mas não existe em `userModel.py` (usar `UserCreate`)
    - Typo em linha 357: `HTTP_200_O0` deveria ser `HTTP_200_OK`
    - Verificar imports nos modelos (redundâncias)